Grouping Similar Things: Cluster Analysis

BDAT 602 – Data Mining and Applications — Lecture 4

Eric Ocran, PhD

Department of Statistics and Actuarial Science
University of Ghana

Outline

What is Cluster Analysis?

Core Question

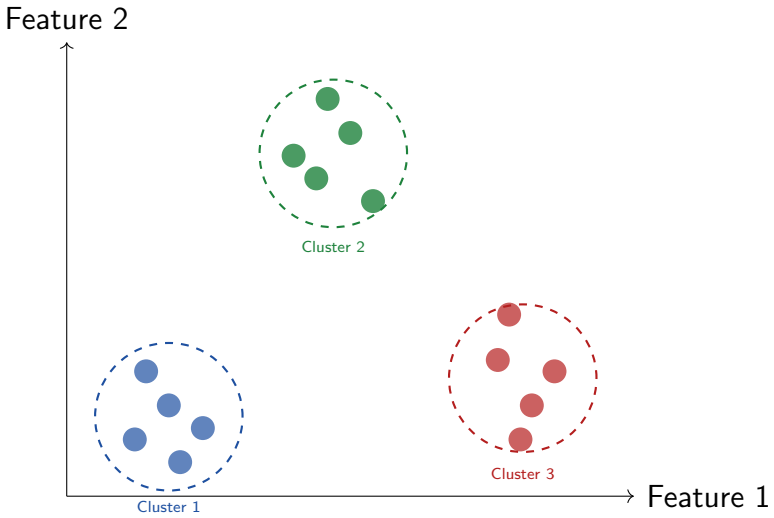
"Which observations in the data are similar enough to each other to be considered members of the same natural group?"

What is Cluster Analysis?

Key characteristics of clustering:

- **Unsupervised**: no predefined labels or target variable — the algorithm discovers groups on its own
- **Exploratory**: we do not know how many groups exist or what they look like in advance.
- **Multiple valid solutions**: clustering has no single correct answer; different algorithms and parameters reveal different structure.
- **Similarity-driven**: observations within a cluster should be as similar as possible to each other, and as different as possible from observations in other clusters.

What is Cluster Analysis?



Clustering vs. Classification: A Critical Distinction

Property	**Classification (Supervised)**	**Clustering (Unsupervised)**
Labels	Known in advance	Not available
Goal	Assign new observations to a known class	Discover unknown groups
Evaluation	Accuracy, AUC, F1 score versus true labels	Silhouette score, within-cluster variance
Output	Predicted class for each observation	Cluster membership and cluster profile
Example	Is this claim fraudulent? (Yes/No)	What types of policyholders exist?
Algorithm	Decision tree, Naïve Bayes, kNN	k-means, DBSCAN, hierarchical clustering

Clustering often precedes classification: discover groups, label them, and then classify new observations.

Clustering vs. Classification: A Critical Distinction

How Clustering Is Used in Practice

Step 1: Cluster policyholders into risk segments (unsupervised).
Step 2: A domain expert examines each cluster and assigns a label ("Low Risk", "High Risk", "Fraud Prone").
Step 3: A classifier is trained to assign new policyholders to these labelled segments.

Clustering Applications in Insurance

Our three business questions this week:

1. **Risk segmentation**: Group the 500,000 policyholders into risk profiles based on age, BMI, chronic conditions, and claim history. Use these segments to guide premium pricing.
   → *k*-means clustering

2. **Behavioural segmentation**: Group policyholders by their engagement behaviour (app logins, support calls, payment method, customer rating) to guide retention strategy.
   → Hierarchical clustering

1. **Anomaly detection via clustering**: Identify policyholders who do not fit any natural cluster — these are potential outliers or fraud cases.
   → DBSCAN

All variables must be **scaled** *before clustering (see Lecture 2).*

Clustering Applications in Insurance

Features used for clustering:

Variable	Role
age	Risk driver
bmi	Risk driver
income	Behavioural
premium	Policy value
num_chronic	Risk driver
num_claims	Claims history
claim_amount	Financial impact
support_calls	Engagement
customer_rating	Satisfaction
app_logins	Engagement

Measuring Similarity: Distance Functions

All clustering algorithms rely on a distance (or similarity) measure. The choice of distance determines which observations are considered "similar".

Euclidean distance (most common):

$$d_E(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{\sum_{j=1}^{p} (x_j - y_j)^2}$$

Minkowski distance (generalisation):

$$d_q(\boldsymbol{x}, \boldsymbol{y}) = \left(\sum_{j=1}^{p} |x_j - y_j|^q \right)^{1/q}$$

$q = 1$: Manhattan; $q = 2$: Euclidean; $q \to \infty$.

Measuring Similarity: Distance Functions

Straight-line distance in p-dimensional space.

Good for: compact, spherical clusters.

Bad for: elongated or irregular clusters.

Manhattan distance (ℓ_1):

$$d_M(x, y) = \sum_{j=1}^{p} |x_j - y_j|$$

Sum of absolute differences along each axis.

More robust to outliers than Euclidean.

Cosine similarity (for text/sparse data):

$$\text{sim}(x, y) = \frac{x^\top y}{\|x\| \|y\|}$$

Measures angle between vectors; ignores magnitude.

Good for: document similarity

Gower distance (mixed data types):

- Handles numeric, ordinal, and nominal variables simultaneously
- Automatically rescales each variable to $[0, 1]$
- Essential when clustering with `plan_tier` + `age` together

Why Scaling is Mandatory Before Clustering

Concrete example with our data:

Two policyholders A and B:

Variable	A	B
age	35 years	40 years
income	\$30,000	\$80,000

Euclidean distance (unscaled):

$$d = \sqrt{(35-40)^2 + (30000-80000)^2} \approx 50{,}000$$

Always Scale Before Clustering

Failure to scale is the single most common mistake in applied cluster analysis. Variables with large ranges dominate the distance computation and make all other variables irrelevant.

Why Scaling is Mandatory Before Clustering

Concrete example with our data:
Income **completely dominates**. The 5-year age difference is invisible.

After Z-score standardisation:

$$d = \sqrt{(-0.36 - 0.00)^2 + (-0.85 - 1.43)^2} \approx 2.31$$

Both variables contribute meaningfully to the distance.

Quick checklist before clustering:

- ✓ All missing values imputed (Lecture 2)
- ✓ All outliers treated (Lecture 2)
- ✓ All variables scaled (Z-score or robust)
- ✓ Categorical variables encoded or Gower distance used
- ✓ Highly correlated variables reduced (PCA optional)

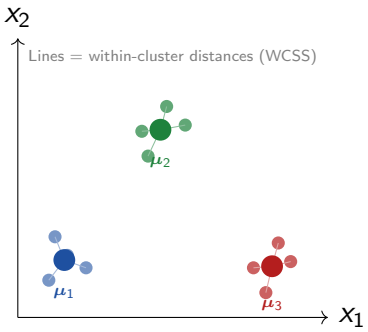
k-Means: The Core Idea

k-Means Objective

Partition n observations into k clusters $\{C_1, C_2, \ldots, C_k\}$ such that the **total within-cluster sum of squares (WCSS)** is minimised:

$$\min_{C_1,\ldots,C_k} \sum_{c=1}^{k} \sum_{\boldsymbol{x} \in C_c} \|\boldsymbol{x} - \boldsymbol{\mu}_c\|^2$$

where $\boldsymbol{\mu}_c = \frac{1}{|C_c|} \sum_{\boldsymbol{x} \in C_c} \boldsymbol{x}$ is the **centroid** of cluster c.



k-Means: The Core Idea

Intuition:

- Each cluster is represented by its centroid (mean)
- Every observation is assigned to the nearest centroid
- We want observations within a cluster to be tightly packed around their centroid
- Minimising WCSS maximises within-cluster cohesion

The Core Idea in simple language

1. For $k = 4$, the algorithm will create 4 clusters.
2. Each cluster has a centre called a centroid.
3. The algorithm repeatedly:
   1. Assigns observations to the nearest centroid.
   2. Recalculates centroids.
   3. Repeats until no further changes occur.
4. **Goal**
   - Make observations inside each cluster very similar to one another.
   - While making different clusters very different from each other.

k-Means Algorithm: Step by Step

Lloyd's Algorithm (standard k-means):

1. **Initialise**: randomly select k observations as initial centroids $\boldsymbol{\mu}_1^{(0)}, \ldots, \boldsymbol{\mu}_k^{(0)}$
2. **Assignment step**: assign each observation $\boldsymbol{x}_i$ to the nearest centroid:

$$c_i^{(t)} = \arg\min_c \|\boldsymbol{x}_i - \boldsymbol{\mu}_c^{(t)}\|^2$$

Important properties:

- Guaranteed to **converge** (WCSS decreases at every step and is bounded below by 0)
- Converges to a **local** minimum, not necessarily the global minimum
- **Sensitive to initialisation**: different random starts can produce very different final clusters
- **Solution**: run multiple random starts (`nstart` in R) and keep the best.

k-Means Algorithm: Step by Step

Lloyd's Algorithm (standard k-means):

1. **Update step**: recompute each centroid as the mean of its assigned observations:

$$\boldsymbol{\mu}_c^{(t+1)} = \frac{1}{|C_c^{(t)}|} \sum_{\boldsymbol{x}_i \in C_c^{(t)}} \boldsymbol{x}_i$$

2. **Repeat** steps 2–3 until cluster assignments no longer change (convergence)

Important properties:

- **k-means++**: smarter initialisation that spreads initial centroids apart; reduces sensitivity to random starts

The k Problem

The algorithm requires you to specify k *in advance*. Choosing the right k is a key challenge — the elbow method and silhouette score help.

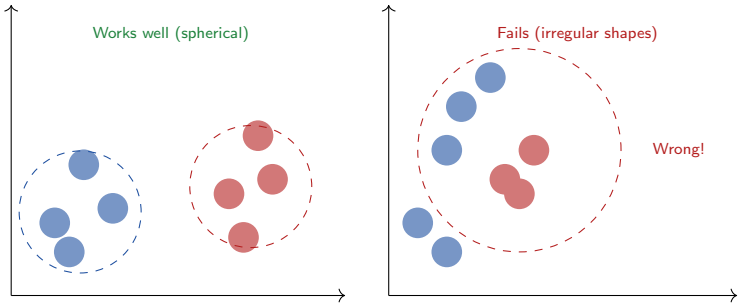
Strengths:

- **Simple and fast**: $O(nkdT)$ where T = iterations; scales well to large n
- **Easy to interpret**: each cluster is described by its centroid
- **Scalable**: Spark's `ml_kmeans()` handles millions of rows
- **Works well** when clusters are compact, convex, and roughly equal in size
- Good first algorithm to try on any clustering problem

Limitations:

- Must specify k in advance
- Sensitive to random initialisation (use `nstart`)
- Assumes **spherical, equally-sized** clusters
- Sensitive to **outliers** (outliers pull centroids)
- Cannot handle **non-convex** or **irregular** shapes
- Results change on each run without a seed

k-Means: Strengths and Limitations



Choosing k: The Elbow Method

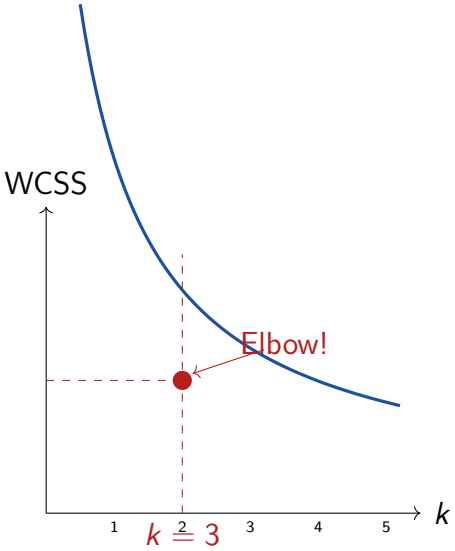
The elbow method:

1. Run k-means for $k = 1, 2, 3, \ldots, K_{\max}$
2. For each k, record the **total within-cluster sum of squares (WCSS)**
3. Plot WCSS vs. k
4. Look for the **"elbow"**: the point where adding another cluster yields diminishing returns in WCSS reduction
5. This elbow suggests the optimal k

Why WCSS always decreases:

- With $k = n$: each observation is its own cluster $\Rightarrow$ WCSS $= 0$
- We want the simplest k that explains most of the structure
- The elbow is where the **rate of decrease** sharply slows

Choosing k: The Elbow Method



R Application: k-Means on the Insurance Dataset

```
library(dplyr)
library(factoextra)
source("simulate_bdat602.R")
health_data <- simulate_bdat602(n = 500000, seed = 602)
# --- Step 1: Select and scale features ---
cluster_vars <- c("age", "bmi", "income", "premium",
"num_chronic_conditions", "num_claims",
"claim_amount", "support_calls",
"customer_rating", "app_logins_monthly")
# Use a 50,000-row sample for local computation
set.seed(602)
health_sub <- health_data |>
filter(!is.na(bmi), !is.na(income)) |>
slice_sample(n = 50000)
```

R Application: *k*-Means on the Insurance Dataset

```
# Scale all selected numeric variables
health_scaled <- health_sub |>
select(all_of(cluster_vars)) |>
mutate(income = log1p(income),
claim_amount = log1p(claim_amount)) |>
scale() |>as.data.frame()
# Confirm: means ~0, sds ~1
colMeans(health_scaled) |> round(3)
apply(health_scaled, 2, sd) |> round(3)
```

R Application: Elbow Method and Running *k*-Means

```
library(factoextra)
# --- Step 2: Elbow method to choose k ---
set.seed(602)
elbow_plot <- fviz_nbclust(
health_scaled,
FUNcluster = kmeans,
method     = "wss",          # within-cluster sum of squares
k.max      = 10,
nstart     = 10) +
labs(title = "Elbow Method: Choosing Optimal k",
x     = "Number of Clusters k",
y     = "Total Within-Cluster Sum of Squares") +
theme_minimal()
elbow_plot
```

R Application: Elbow Method and Running *k*-Means

```
# --- Step 3: Fit k-means with chosen k (e.g. k = 4) ---
set.seed(602)
km_fit <- kmeans(
health_scaled,centers = 4,
nstart  = 25,       # 25 random starts; keep best
iter.max = 100      # allow enough iterations to converge
)
# Cluster sizes
km_fit$size
# Within-cluster SS and between-cluster SS
km_fit$tot.withinss
km_fit$betweenss / km_fit$totss   # proportion of variance
   explained
```

R Application: Profiling and Visualising Clusters

```r
library(ggplot2)
library(factoextra)
# --- Step 4: Add cluster labels back to original data ---
health_sub$cluster <- factor(km_fit$cluster)
# --- Step 5: Profile each cluster ---
health_sub |> group_by(cluster) |>
summarise(
n              = n(),
avg_age        = mean(age),
avg_bmi        = mean(bmi,    na.rm = TRUE),
avg_income     = mean(income, na.rm = TRUE),
avg_claims     = mean(num_claims),
avg_claim_amt  = mean(claim_amount),
pct_smoker     = mean(smoker) * 100,
pct_fraud      = mean(fraud_flag) * 100,
pct_churned    = mean(churned)    * 100
) |>arrange(desc(avg_claim_amt))
```

R Application: Profiling and Visualising Clusters

```
library(ggplot2)
library(factoextra)
# --- Step 6: Visualise clusters in 2D (PCA projection) ---
fviz_cluster(km_fit,
data        = health_scaled,
geom        = "point",
ellipse     = TRUE,
ellipse.type= "convex",
alpha       = 0.3,
palette     = c("#E41A1C","#377EB8","#4DAF4A","#984EA3"),
ggtheme     = theme_minimal(),
main        = "k-Means Clusters (k=4): PCA Projection")
```

Interpreting Cluster Profiles: Example

Cluster	Avg Age	Avg BMI	Avg Claims	% Smoker	Label
1	58	31	3.8	42%	**High Risk**
2	34	23	0.4	8%	**Low Risk**
3	45	27	1.9	21%	**Moderate Risk**
4	62	25	1.1	12%	**Elderly, Healthy**

Interpreting Cluster Profiles: Example

Reading the profiles:

- **Cluster 1** (High Risk): older, obese, high claim frequency, many smokers — highest premium pricing tier
- **Cluster 2** (Low Risk): young, healthy weight, almost no claims, few smokers — preferred customer segment
- **Cluster 3** (Moderate): middle-aged, some risk factors, moderate claims
- **Cluster 4** (Elderly Healthy): old but low BMI and few claims — long-term, stable customers

Business actions:

- **Cluster 1**: increase premiums; offer wellness incentives; prioritise for case management
- **Cluster 2**: offer loyalty discounts; upsell add-on riders; cross-sell life insurance
- **Cluster 4**: design age-appropriate products; prioritise retention (high lifetime value)

Labels are assigned by a **domain expert**, not the algorithm.

Hierarchical Clustering: The Big Idea

Key Difference from k-Means

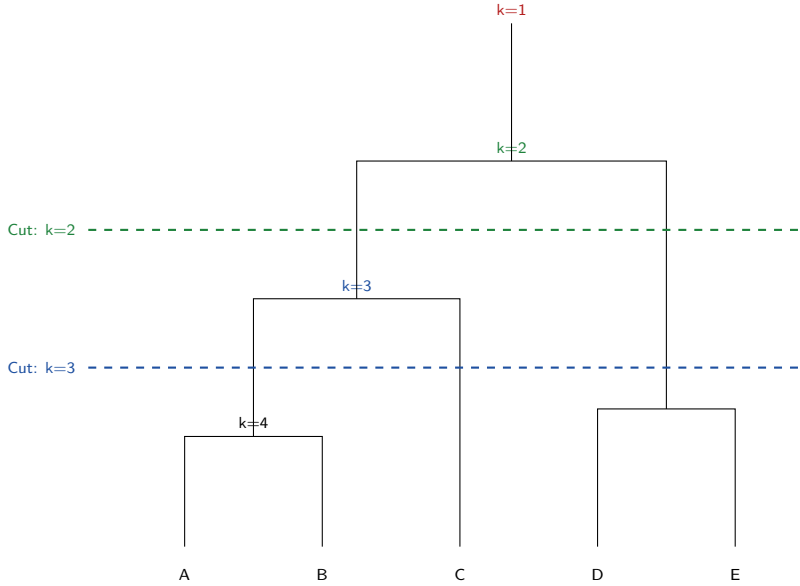
Hierarchical clustering does **not** require specifying k in advance. Instead, it builds a **full tree of nested clusters** (a *dendrogram*) from which you can cut at any level to obtain any number of clusters.

Two approaches:
Divisive (top-down):

- Start: all observations in one cluster
- Repeatedly **split** the largest cluster
- Less common; computationally expensive

Two approaches:

1. **Agglomerative** (bottom-up) — most common:
   - Start: each observation is its own cluster (n clusters)
   - Repeatedly **merge** the two most similar clusters
   - Stop: all observations are in one cluster
   - Cut the dendrogram at a chosen height to get k clusters


k=1
k=2
Cut: k=2
k=3
Cut: k=3
k=4
A
B
C
D
E

Linkage Methods: How Clusters Are Merged

At each step, the two "closest" clusters are merged. **Linkage** defines what "closest" means between two clusters (not just two points).

Single linkage (minimum):

$$d(C_a, C_b) = \min_{\boldsymbol{x} \in C_a,\, \boldsymbol{y} \in C_b} d(\boldsymbol{x}, \boldsymbol{y})$$

Distance between the two *nearest* points in each cluster.

Prone to "chaining": creates long, strung-out

Average linkage (UPGMA):

$$d(C_a, C_b) = \frac{1}{|C_a||C_b|} \sum_{\boldsymbol{x} \in C_a} \sum_{\boldsymbol{y} \in C_b} d(\boldsymbol{x}, \boldsymbol{y}$$

Average distance between all pairs. Compromise between single and complete.

Ward's linkage (most popular):

$$d(C_a, C_b) = \frac{|C_a||C_b|}{|C_a| + |C_b|} \|\boldsymbol{\mu}_a - \boldsymbol{\mu}_b\|^2$$

Linkage Methods: How Clusters Are Merged

Complete linkage (maximum):

$$d(C_a, C_b) = \max_{\boldsymbol{x} \in C_a,\, \boldsymbol{y} \in C_b} d(\boldsymbol{x}, \boldsymbol{y})$$

Distance between the two *farthest* points.

Tends to produce compact, roughly equal-sized clusters.

Average linkage (UPGMA):
Merge two clusters that produce the *smallest increase in total WCSS*.

Default choice: produces compact, well-separated clusters; analogous to k-means.

Recommendation

Use **Ward's linkage** as the default. Switch to complete linkage if clusters are very different in size.

R Application: Hierarchical Clustering and Dendrogram

```r
library(factoextra)
library(ggplot2)
# --- Use a smaller subset for hierarchical clustering ---
# hclust() requires an n x n distance matrix: expensive for #large
    n
set.seed(602)
health_hc <- health_scaled |> slice_sample(n = 2000)
# --- Step 1: Compute distance matrix ---
dist_matrix <- dist(health_hc, method = "euclidean")
# --- Step 2: Hierarchical clustering with Ward's linkage ---
hc_fit <- hclust(dist_matrix, method = "ward.D2")
```

R Application: Hierarchical Clustering and Dendrogram

```
# --- Step 3: Plot the dendrogram ---
fviz_dend(hc_fit,
k = 4, cex = 0.4, palette = "jco", rect = TRUE,
rect_fill  = TRUE,
main       = "Dendrogram: Health Insurance Policyholders",
xlab       = "Policyholders",
ylab       = "Height (Ward Linkage Distance)"
        )
# --- Step 4: Cut the tree at k=4 ---
hc_labels <- cutree(hc_fit, k = 4)
table(hc_labels)
# --- Compare with k-means solution ---
# Are the 4 clusters from hclust similar to k-means?
table(km_fit$cluster[1:2000], hc_labels)
```

Reading a Dendrogram

How to read a dendrogram:

- Each **leaf** at the bottom represents one observation (or a small cluster in large datasets)
- The **height** of a merge (the y-axis) represents the distance between the two clusters being merged — higher merges = more dissimilar clusters

Choosing the cut height:

- Look for the **largest vertical gap** in the dendrogram — cutting here gives the most natural grouping
- Also use the silhouette score (Section 5) to validate the choice
- A dendrogram with no large gaps suggests the data has no strong cluster structure

Reading a Dendrogram

How to read a dendrogram:

- **Long vertical branches** before a merge indicate that the merge joins two genuinely different groups — a good place to cut
- **Short branches** suggest observations or clusters that are very similar to each other

Scalability Warning

`hclust()` requires computing and storing an $n \times n$ distance matrix. For $n = 500{,}000$: memory needed $\approx 500{,}000^2 \times 8$ bytes $= 2$ petabytes. **Use only on subsamples** ($n \leq 5{,}000$).

DBSCAN: Motivation

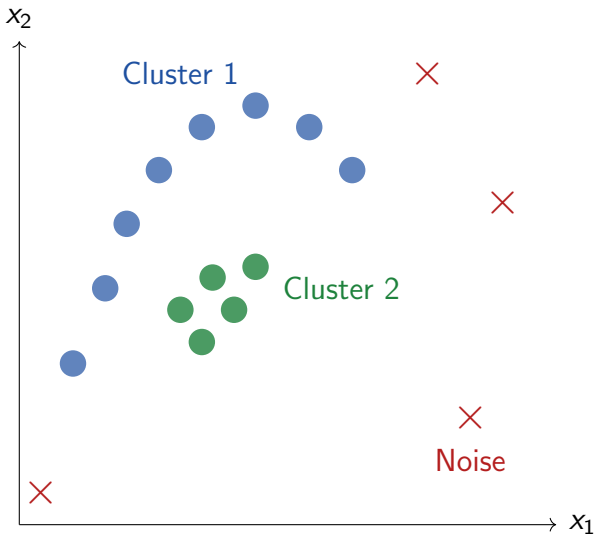
The problem with k-means and hierarchical clustering:

- Both assume clusters are **convex and roughly spherical**
- Both are **sensitive to outliers**: a single extreme point can be assigned to a cluster and distort its centroid
- Neither can detect clusters of **arbitrary shape** (crescent, ring, spiral)
- Neither has a built-in concept of **noise** (observations that belong to no cluster)

DBSCAN Solution

Density-Based Spatial Clustering of Applications with Noise defines clusters as **dense regions of points** separated by regions of lower density. Points in sparse regions are labelled as **noise** — they belong to no cluster.

DBSCAN: Motivation



DBSCAN: Core Concepts and Algorithm

Two parameters:

- ε (eps): the radius of the neighbourhood around each point
- MinPts: the minimum number of points required within ε to define a dense region

Three point types:

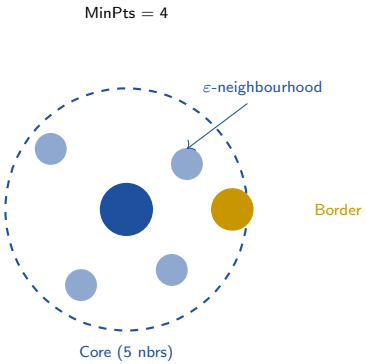
1. **Core point**: has $\geq$ MinPts points within its ε-neighbourhood (including itself).Has many nearby neighbours.
2. **Border point**: within the ε-neighbourhood of a core point but has $<$ MinPts neighbours itself. Near a core point but not dense enough itself.
3. **Noise point**: not within ε of any core point; assigned label -1.Does not belong to any cluster.

DBSCAN: Core Concepts and Algorithm

Algorithm:

1. For each unvisited point p: find all points within ε
2. If $|\text{neighbours}(p)| \geq \text{MinPts}$: start a new cluster; expand by adding all density-reachable points
3. If not: mark p as noise (may be reassigned later as a border point)

DBSCAN: Core Concepts and Algorithm



DBSCAN: Choosing ε and MinPts

Choosing MinPts:

- Rule of thumb: MinPts $\geq p + 1$ where p = number of features
- For 2D data: MinPts $= 4$ or 5 is common
- Higher MinPts $\rightarrow$ more noise points; denser clusters required
- For noisy data: use larger MinPts

Choosing ε — the k-NN distance plot:

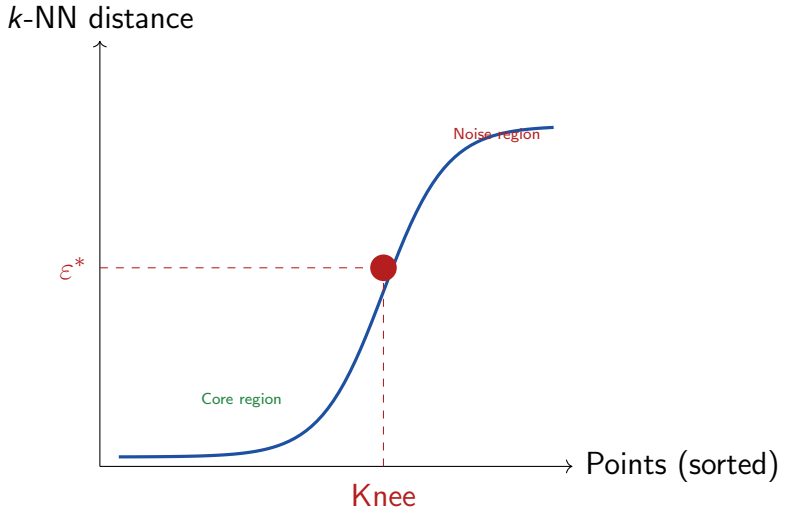
1. For each point, compute its distance to its kth nearest neighbour (where $k = \text{MinPts} - 1$)
2. Sort these distances and plot them

DBSCAN: Choosing ε and MinPts

Choosing ε — the k-NN distance plot:

1. Look for the **"knee"** in the plot: this is the natural threshold separating core points (small k-NN distance) from noise (large k-NN distance). Set ε at the knee value

DBSCAN: Choosing ε and MinPts



R Application: DBSCAN on Insurance Data

```r
library(dbscan)
library(factoextra)
# --- Use a 2D projection for easy visualisation ---
# Apply PCA first; cluster on PC1 and PC2
pca_result <- prcomp(health_scaled, scale. = FALSE)
pca_2d     <- pca_result$x[, 1:2]   # first two PCs
# --- Step 1: k-NN distance plot to choose eps ---
kNNdistplot(pca_2d,k    = 4,          # MinPts - 1
main = "k-NN Distance Plot: Choosing epsilon"
)
abline(h = 0.5, col = "red", lty = 2)   # candidate eps
# --- Step 2: Run DBSCAN ---
db_fit <- dbscan(pca_2d,
eps     = 0.5,     # set from k-NN distance plot
minPts  = 5
)
```

R Application: DBSCAN on Insurance Data

```
# Cluster membership (0 = noise)
table(db_fit$cluster)
# --- Step 3: Visualise DBSCAN results ---
fviz_cluster(db_fit,
data     = pca_2d,
geom     = "point",
palette  = "Set2",
ggtheme  = theme_minimal(),
main     = "DBSCAN Clusters (eps=0.5, MinPts=5)"
)
```

R Application: Using DBSCAN for Anomaly Detection

```
# --- DBSCAN noise points as anomaly candidates ---
# Points labelled 0 by DBSCAN do not belong to any dense cluster
# In our insurance context, these may be fraudulent or unusual
   records
noise_idx <- which(db_fit$cluster == 0)
cat("Number of noise points detected:", length(noise_idx), "\n")
cat("As % of total:", round(100 * length(noise_idx)/nrow(pca_2d),
   2), "%\n")
# Profile the noise points
health_sub[noise_idx, ] |>
summarise(n  = n(),avg_age = mean(age),
avg_bmi = mean(bmi, na.rm = TRUE),
avg_claim  = mean(claim_amount),
pct_fraud  = mean(fraud_flag)  * 100,
pct_unemployed= mean(employment_type == "Unemployed") * 100
)
```

R Application: Using DBSCAN for Anomaly Detection

```
# Compare fraud rate: noise points vs.\ cluster members
cat("Fraud rate in noise points: ",
round(mean(health_sub$fraud_flag[noise_idx]) * 100, 1), "%\n")
cat("Fraud rate in clusters    : ",
round(mean(health_sub$fraud_flag[-noise_idx]) * 100, 1), "%\n")
```

Key Finding

If fraud rate among DBSCAN noise points is significantly higher than among cluster members, DBSCAN is effectively acting as an unsupervised fraud detector — without ever seeing the `fraud_flag` label.

How Do We Know if Our Clusters Are Good?

The challenge of evaluating clustering:

- There is no "ground truth" label to compare against (unlike classification)
- We need **internal validation measures** — metrics computed from the data itself, without external labels
- The most widely used: the **silhouette score**

Interpreting $s(i)$:

$s(i)$	**Interpretation**
$\approx +1$	Excellent clustering
≈ 0	On boundary between clusters
≈ -1	Probably in the wrong cluster

How Do We Know if Our Clusters Are Good?

Silhouette Score for Observation i

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}}$$

where:

- $a(i)$ = mean distance from i to all other points **in the same cluster** (cohesion)
- $b(i)$ = mean distance from i to all points **in the nearest other cluster** (separation)

Overall silhouette score: mean of $s(i)$ across all observations.

Score	Quality
> 0.70	Strong structure
0.50–0.70	Reasonable structure
0.25–0.50	Weak structure
< 0.25	No substantial structure

Silhouette Score: Geometric Intuition

Understanding $a(i)$ and $b(i)$:

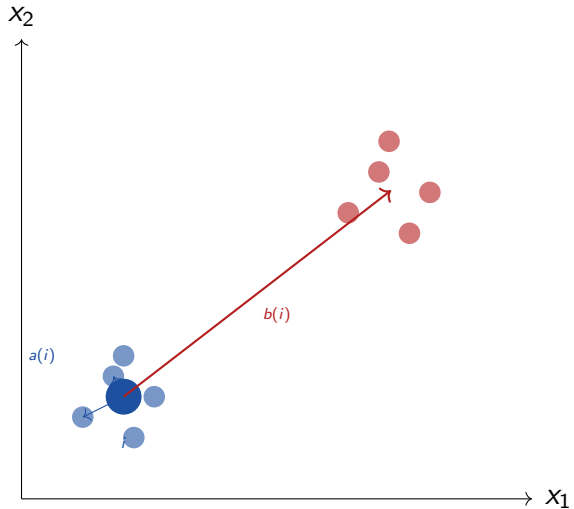
Consider observation i in Cluster 1:

- $a(i)$: average distance from i to all other members of Cluster 1
  $\rightarrow$ measures how **tight** the cluster is around i
- $b(i)$: for each other cluster, compute the average distance from i to that cluster's members; take the **minimum**
  $\rightarrow$ measures how far i is from the nearest alternative cluster

A high silhouette score requires:

- $a(i)$ is **small**: i is close to its cluster-mates
- $b(i)$ is **large**: i is far from the nearest alternative cluster
- The cluster containing i is well-separated from its neighbours

Silhouette Score: Geometric Intuition



R Application: Silhouette Score and Optimal k

```
library(cluster)
library(factoextra)
# --- Method 1: Silhouette plot for a fixed k ---
# Compute pairwise distances
dist_sub <- dist(health_scaled[1:5000, ], method = "euclidean")
# k-means with k=4
set.seed(602)
km4 <- kmeans(health_scaled[1:5000, ], centers = 4, nstart = 25)
# Silhouette values for each observation
sil_scores <- silhouette(km4$cluster, dist_sub)
summary(sil_scores)           # mean silhouette width
# Silhouette plot
fviz_silhouette(sil_scores,palette = "jco",
ggtheme = theme_minimal(),
main    = "Silhouette Plot: k-Means (k=4)"
)

                # --- Method 2: Silhouette method to choose optimal
```

R Application: Silhouette Score and Optimal k

```
# --- Method 2: Silhouette method to choose optimal k ---
fviz_nbclust(
health_scaled[1:5000, ],
FUNcluster = kmeans,
method     = "silhouette",
k.max      = 10,
nstart     = 10
) +
labs(title = "Silhouette Method: Optimal Number of Clusters",
x     = "Number of Clusters k",
y     = "Average Silhouette Width") +
theme_minimal()
```

Other Cluster Validation Measures

Measure	What It Measures	Range	Better
Silhouette	Cohesion and separation	$[-1, +1]$	Higher
WCSS / Inertia	Within-cluster tightness	$[0, \infty)$	Lower
Dunn Index	Minimum inter-cluster distance divided by maximum intra-cluster distance	$(0, \infty)$	Higher
Davies–Bouldin	Average similarity between each cluster and its most similar cluster	$[0, \infty)$	Lower
Calinski–Harabasz	Ratio of between-cluster to within-cluster dispersion	$(0, \infty)$	Higher
External measures (require known labels):			
Rand Index	Agreement between clustering and true labels	$[0, 1]$	Higher
Adjusted Rand	Rand index corrected for chance	$[-1, 1]$	Higher

Other Cluster Validation Measures

Practical Advice

Use the **silhouette score** and the **elbow plot** together. If they agree on k, proceed with confidence. If they disagree, try both k values and profile the clusters to see which makes more business sense.

Scaling k-Means to 500,000 Rows with Spark

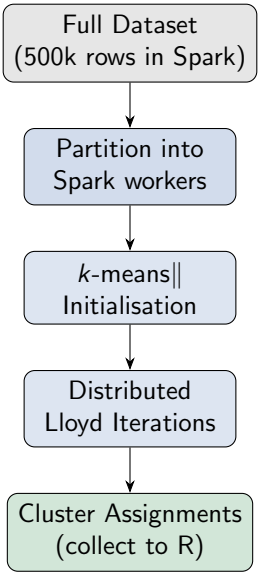
Why local k-means cannot handle our full dataset:

- Base R `kmeans()` loads all data into RAM
- 500,000 rows $\times$ 10 features $\times$ 8 bytes $\approx$ 40 MB for data; but with `nstart=25`, intermediate storage multiplies
- More importantly: with millions of rows in industry settings, even 40MB scales to gigabytes
- Spark's `ml_kmeans()` uses a **distributed, mini-batch** implementation of k-means that keeps data in Spark and only returns cluster assignments.

Spark k-Means: How It Works

Spark uses the k-**means**|| (k-means parallel) initialisation — a scalable variant of k-means++ — followed by standard Lloyd iterations distributed across the Spark cluster.

Scaling k-Means to 500,000 Rows with Spark



R Application: Distributed k-Means with `sparklyr`

```
library(sparklyr)
library(dplyr)
sc <- spark_connect(master = "local[*]", version = "3.4.1")
# --- Step 1: Copy data to Spark and preprocess ---
health_tbl <- copy_to(sc, health_data,
name = "health_ins", overwrite = TRUE)
# Scale in Spark using ft_standard_scaler (via pipeline)
health_spark_scaled <- health_tbl |>
mutate(log_income = log1p(income),
log_claim  = log1p(claim_amount)) |>
select(age, bmi, log_income, premium,
num_chronic_conditions, num_claims,
log_claim, support_calls,
customer_rating, app_logins_monthly)
```

R Application: Distributed *k*-Means with `sparklyr`

```
library(sparklyr)
library(dplyr)
# --- Step 2: Run k-means in Spark ---
set.seed(602)
spark_km <- health_spark_scaled |>ml_kmeans(
formula    = ~ .,
k          = 4,
max_iter   = 100,
init_mode  = "k-means||",   # scalable initialisation
seed       = 602
)
# --- Step 3: Inspect cluster centres ---
spark_km$centers
```

R Application: Collecting and Profiling Spark Clusters

```
# --- Step 4: Get predictions (cluster assignments) ---
spark_predictions <- ml_predict(spark_km, health_spark_scaled)

# --- Step 5: Join back to full data for profiling ---
# Add record_id to spark data for joining
health_with_id <- health_tbl |>
mutate(
log_income = log1p(income),
log_claim  = log1p(claim_amount)) |>
select(record_id, age, bmi, log_income, premium,
num_chronic_conditions, num_claims,
log_claim, support_calls,
customer_rating, app_logins_monthly,
smoker, fraud_flag, churned, plan_tier)
cluster_results <- ml_predict(spark_km, health_with_id)
```

R Application: Collecting and Profiling Spark Clusters

```
# --- Step 6: Profile clusters in Spark ---
cluster_results |>
group_by(prediction) |>
summarise(
n              = n(),
avg_age        = mean(age,           na.rm = TRUE),        avg_bmi
         = mean(bmi,           na.rm = TRUE),
avg_claims     = mean(num_claims,    na.rm = TRUE),
pct_fraud      = mean(fraud_flag,    na.rm = TRUE) * 100,
pct_churned    = mean(churned,       na.rm = TRUE) * 100,
avg_support    = mean(support_calls,na.rm = TRUE)
) |>
collect() |>
arrange(desc(avg_claims))
```

R Application: Elbow Method in Spark

```
# --- Elbow method in Spark: run k-means for k=2..8 ---
wcss_results <- purrr::map_dfr(2:8, function(k) {
model <- health_spark_scaled |>
ml_kmeans(
formula    = ~ .,
k          = k,
max_iter   = 50,
init_mode  = "k-means||",
seed       = 602)
tibble(k    = k,
wcss = model$summary$trainingCost  # within-cluster SS
)
})
```

R Application: Elbow Method in Spark

```
# Plot the elbow curve
library(ggplot2)
ggplot(wcss_results, aes(x = k, y = wcss)) +
geom_line(colour = "steelblue", linewidth = 1) +
geom_point(colour = "steelblue", size = 3) +
labs(title = "Elbow Method: Spark k-Means (n = 500,000)",
x     = "Number of Clusters k",
y     = "Within-Cluster Sum of Squares (WCSS)"
) +theme_minimal()
```

Spark Elbow Method

Running the elbow loop in Spark is feasible because each `ml_kmeans()` call executes in distributed fashion. On a local Spark session this still takes some minutes; on a multi-node cluster it runs in seconds per k.

What We Covered Today

Conceptual foundations:

- ✓ Clustering vs. classification: unsupervised discovery vs. supervised prediction
- ✓ Distance measures: Euclidean, Manhattan, Minkowski, Cosine, Gower
- ✓ Why scaling is mandatory before any distance-based algorithm
- ✓ k-means: WCSS objective, Lloyd's algorithm, elbow method, k-means++

Practical (R):

- ✓ Data preparation: scaling, log-transforming skewed variables
- ✓ Elbow method with `fviz_nbclust()`
- ✓ Running k-means with `kmeans()` and multiple starts
- ✓ Profiling clusters: `group_by()` summaries
- ✓ Cluster visualisation with `fviz_cluster()`

What We Covered Today

Conceptual foundations:

- ✓ k-means strengths (fast, scalable) and limitations (spherical clusters, sensitive to outliers)
- ✓ Hierarchical clustering: agglomerative vs. divisive; dendrogram reading; linkage methods (single, complete, average, Ward's)
- ✓ DBSCAN: core/border/noise points, ε and MinPts, k-NN distance plot
- ✓ Silhouette score: cohesion, separation, interpretation; other validation measures.

Practical (R):

- ✓ Hierarchical clustering with `hclust()`, Ward's linkage, `cutree()`
- ✓ Dendrogram visualisation with `fviz_dend()`
- ✓ DBSCAN with `dbscan()`, k-NN distance plot, noise point analysis
- ✓ Silhouette scoring with `silhouette()` and `fviz_silhouette()`

Looking Ahead: Lecture 5

Next Week: Predicting Categories — Classification and Model Evaluation

Having discovered natural groups (clustering), we now ask a supervised question: given a labelled training set, can we build a model that correctly assigns new policyholders to categories — high risk, churned, or fraudulent?

Key Takeaways

Three Rules of Cluster Analysis

1. **Always scale your data before clustering.** Failure to scale means variables with large ranges dominate the distance computation. Use Z-score standardisation and log-transform right-skewed variables.
2. **No algorithm is universally best.** k-means is fast and scalable but assumes spherical clusters. Hierarchical clustering reveals nested structure but does not scale. DBSCAN finds arbitrary shapes and identifies noise but requires careful parameter tuning.
3. **Cluster labels are assigned by domain experts, not algorithms.** The algorithm assigns numbers (1, 2, 3). A human with business knowledge assigns the meaning ("Low Risk", "High Risk"). Always profile clusters before naming them.

Key Takeaways

Algorithm	**Shapes**	**Scalable?**	**Handles Noise?**
k-means	Spherical only	Yes (Spark)	No
Hierarchical	Any (small n)	No ($O(n^2)$)	No
DBSCAN	Arbitrary	Moderate	Yes

Self-Study and Further Reading

Core reading for Lecture 4:

- Han, Kamber & Pei (2011), *Data Mining*:
  Ch. 10: Cluster Analysis: Basic Concepts and Methods
  Ch. 11: Advanced Cluster Analysis
- Tan, Steinbach & Kumar (2019):
  Ch. 7: Cluster Analysis: Basic Concepts and Algorithms
  Ch. 8: Cluster Analysis: Additional Issues and Algorithms
- Leskovec et al. (2020), *MMDS*:
  Ch. 7: Clustering (free at `mmds.org`)

R resources:

- `factoextra` vignette:
  `cran.r-project.org/package=fac`
- `dbscan` package documentation:
  `cran.r-project.org/package=dbs`
- `sparklyr` ML guide:
  `spark.rstudio.com/guides/mllib`

Practice exercises:

- Run k-means with $k = 2, 3, 4, 5$ on the insurance data; use both the elbow plot and silhouette method to choose the best k; do they agree?
- Run DBSCAN on the 2D PCA projection; compare the fraud rate among noise points vs. cluster members
- Use `sparklyr::ml_kmeans()` on the full 500,000 rows; profile the 4 clusters and assign meaningful business labels

Questions?

Next: Lecture 5 — Predicting Categories:
Classification and Model Evaluation

BDAT 602 · Data Mining and Applications · University of Ghana